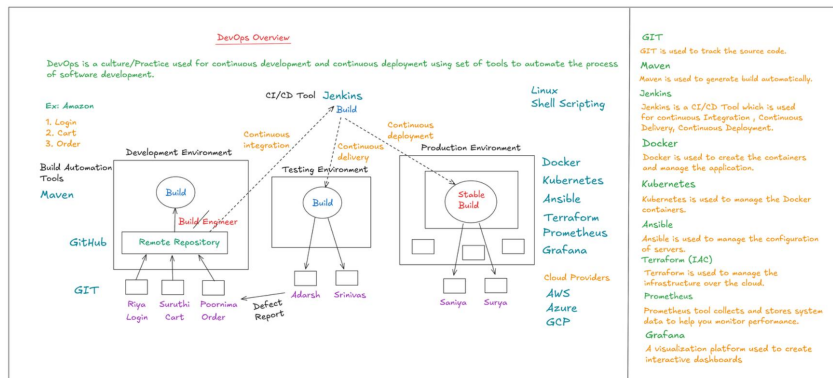


Installation and setup of Jenkins

- 1) Go to any browser and write download jenkins.
- 2) Select the first link which says "Download and deploy"
- 3) Scroll down and select Windows. It will start downloading.
- 4) Go to the downloaded location and install jenkins.
- 5) Click on next ---> next ----> Select "Run service as local system"
-----> click on Test Port and click on next.
- 6) Here it will ask for a path. Go to Local disk C ---> Program files
-----> Java ----> jdk-21 / jdk-17 . Copy that path it will look something like this: **C:\Program Files\Java\jdk-21**
- 7) Paste this path and click on next and then install.
- 8) Go to a browser and type: **localhost:8080**
- 9) Once you type this a new tab will be opened and you will get a path like this:
copy the path and paste it in navigation bar of the downloads.
- 10) Here it will ask for the way to open, select text file and you will get a password which looks like this: **3df712c2bd86400barvea1c0f8c16fe3**
- 11) Paste this password in the browser where it asked.
- 12) After pasting the password you will get two options: "Install suggested plugins" and "Select plugins to install" select "Install suggested plugins" option.
- 9) After that you will be asked to fill all the details (username, password, etc). Give all the details and then click on Save and Continue. Then click on Save and Finish and click on Start using Jenkins.

Jenkins

It is a free and open source CI/CD Tool which automates the process of building, testing and deploying the software into production server.



Continuous integration (CI)

Continuously collecting the source code from GitHub and generating the build. This process is called as continuous integration.

Continuous Delivery (CD)

Continuous Delivery, we are preparing the build ready to be deployed into production server or client server.

Continuous Deployment (CD)

Continuous Deployment is an extension of Continuous Delivery, where the entire process is fully automated.

Here we are continuously installing the build into production server or client server.

History

2004: Kohsuke Kawaguchi created a project called as "Hudson" to automate the process of building, and testing the software. He was working in sun microsystem.

2010: Oracle acquired sun microsystem which created conflict between hudson and oracle.

2011: Hudson project was renamed as "Jenkins"

Features of Jenkins:

1. Free and open source
2. It supports extensible plugins
3. It supports pipeline as a code.
4. It is user friendly
5. It is scalable because it supports master-slave architecture.
6. It supports authentication (username and password) and authorization (Permission)
7. It can be integrated with third-party application

Alternation Tools:

1. GitHub Actions
2. Gitlab CI/CD
3. Teamcity
4. Bamboo
5. Travis CI
6. Circle CI

Jobs in Jenkins

A job is a specific task performed in Jenkins and through the job we can automate the process of generating build, testing, and deploying the projects.

Types of Jobs in Jenkins

1. Freestyle in Job:

- > A Freestyle Project is the most basic and flexible type of Jenkins job.
- > It allows you to define simple build, test and deploy steps directly through the Jenkins graphical interface - no code or Jenkinsfile required.
- > It is a type of job which doesn't require any additional installation of tools where as in Freestyle project, all the tools are pre-installed. While performing the job we just need to do manual configuration for setting up the job.

Steps to perform Java project in Freestyle job in Jenkins:

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Freestyle project" then click on OK.
- 3) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/2894-vaishu/java.git>
- 4) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 5) Scroll down and under "Build Steps" click on Add build step under that choose "Execute Windows batch command" option. Then write these two commands:
 - `javac Test.java` → This command is used for compiling Java project
 - `java Test.java` → This command is used for executing Java project
- 6) Scroll down and finally click on Apply and then click on Save.
- 7) In the side bar you have an option called as "Build Now" click on that. You can see a build is being generated click on that once after it is generated.

Workspace in Jobs:

- > In Jenkins, a workspace is the directory on the Jenkins server (or agent) where Jenkins checks out your project files, performs build operations, and stores temporary data while a job is running.
- > In simple terms: The workspace is the "working folder" Jenkins uses for a job's execution.
- > When a Jenkins job (Freestyle, Maven, or Pipeline) runs, Jenkins creates a workspace for that job. Inside that workspace, it places the source code, build scripts, artifacts, and any intermediate files needed during the build process.

Steps to perform Maven project in Freestyle job in Jenkins:

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Freestyle project" then click on OK.
- 3) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/2894-vaishu/mavenproject.git>
- 4) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 5) Scroll down and under "Build Steps" click on Add build step under that choose "Execute Windows batch command" option. Then write this command: `mvn package`
- 6) Scroll down and finally click on Apply and then click on Save.
- 7) In the side bar you have an option called as "Build Now" click on that. You can see a build is being generated click on that once after it is generated.

2. Maven Project Job:

- > A Maven Project job type is a specialized Jenkins job designed specially for Apache Maven-based Java projects.
- > It understands Maven's pom.xml file and automatically handles build lifecycle (compile, test, package, deploy, etc).

Steps to create job using maven project job in Jenkins:

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Maven project" then click on OK.
- 3) If it is not visible then follow this step:
In the top right corner you have an option called as "Manage Jenkins" click on that. Once you are inside, under "System Configuration" select the "Plugins" option. In the sidebar you have "Available plugins" option select that and in the searchbar write "Maven Integration" and select the plugin and give Install.
- 4) Here after installing Maven plugin, you need to install one more tool. In the top right corner you have an option called as "Manage Jenkins" click on that. Once you are inside, under "System Configuration" select the "Tools" option. Select that and scroll down till you find "Maven installations". There click on "Add Maven", under which give any name you want and untick the "Install automatically" option. Under "MAVEN_HOME" copy the path where you have installed maven locally to your system. It looks something like this: `C:\Program Files\apache-maven-3.9.11` Then click on Apply and then click on Save.
- 5) After doing all this once again select option called as "New Item" click on it. Give a job name and below select the option called as "Maven project" then click on OK.
- 6) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/Sumukha47/mavenproject.git>
- 7) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 8) Scroll down and under "Triggers" you have an option "Build whenever a SNAPSHOT dependency is built" untick that. It is optional.
- 9) Scroll down and under "Build" you get an option "Goals and options" just give the goal. Example: If you want to create a package don't give `mvn package` instead give `package` You can give multiple goals and options in this format: `package verify site`
- 10) Scroll down and finally click on Apply and then click on Save.
- 11) In the side bar you have an option called as "Build Now" click on that. You can see a build is being generated click on that once after it is generated.

Build Trigger

- > A build Trigger in Jenkins is an event or condition that automatically starts (triggers) a build job.
- > Instead of manually clicking "Build now" triggers let jenkins start jobs automatically

Type of Build Triggers:

1. Build Periodically:

- > The build periodically trigger is used to schedule jenkins job to run automatically at specific times, dates, or interval.
- > It is used to execute a job on a fixed schedule, even if there are no changes in the source code or other inputs.

Cron Syntax Format: **MINUTE HOUR DOM MONTH DOW**

where:

- > MINUTE: 0-59
- > HOUR: 0-23
- > DOM: Day of Month (1-31)
- > MONTH: 1-12
- > DOW: Day of Week (0-7, where 0 and 7 represent Sunday)
- > Asterisk * means "every possible value" for that field.

Example:

1. If you write * * * * *
 - * in the minute field --> every minute
 - * in the hour field --> every hour
 - * in the day of month --> every day of the month
 - * in the month field --> every month
 - * in the day of week field --> every day of the week
 - > so overall, this means : "Run the job all the time"
2. If you write : 0 * * * *
 - > It means jenkins runs the job every hour, at minute 0 (12:00, 1:00, 2:00)
3. If you write : 0 2 * * *
 - > It means the job will run once everyday at 2am
4. 0 2 1 * *
 - Run at 2 am on 1st of every month
5. 0 2 1,5,10,15 * *
 - Run at 2 am on 1st,5th,10,15 of every month
6. 0 2 1 1,7 *
 - run at 2am on 1st of jan, july
7. 0 2 * * 1-5
mon-fri
8. */5 * * * *
9. 0 */2 * * *

Steps to run the build using Build periodically in Windows:

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Freestyle project" then click on OK.
- 3) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/2894-vaishu/java.git>
- 4) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 5) Scroll down and you will get a section called as "Triggers" under that select "Build periodically". As soon as you select this it will ask you to write your schedule, write any cron schedule as you want.
- 6) Scroll down and under "Build Steps" click on Add build step under that choose "Execute Windows batch command" option. Then write these two commands:
 - javac Test.java** —> This command is used for compiling java project
 - java Test.java** —> This command is used for executing java project
- 7) Scroll down and finally click on Apply and then click on Save.
- 8) Here no need to click on "Build Now" it will automatically generate a build based on the schedule you gave.

2. Poll SCM:

--> The "Poll SCM" trigger makes Jenkins periodically check the source code repository to detect if there are new commits or changes.

--> It is used to automatically trigger a build when new code is detected in the repository.

Steps to run the build using Poll SCM in Windows:

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Freestyle project" then click on OK.
- 3) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/2894-vaishu/java.git>
- 4) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 5) Scroll down and you will get a section called as "Triggers" under that select "Poll SCM". As soon as you select this it will ask you to write your schedule, write any cron schedule as you want.
- 6) Scroll down and under "Build Steps" click on Add build step under that choose "Execute Windows batch command" option. Then write these two commands:
`javac Test.java` —> This command is used for compiling java project
`java Test.java` —> This command is used for executing java project
- 7) Scroll down and finally click on Apply and then click on Save.
- 8) There is no need to click on "Build Now" it will automatically generate the first build and then if there is any change in the GitHub (new commit, updated branch, etc.) only then the second build is generated based on the schedule you gave.

3. GitHub Hook Trigger for GIT SCM Polling

--> This trigger enables Jenkins to start builds immediately when GitHub detects changes

--> It is used to provide realtime build instead of time-based polling.

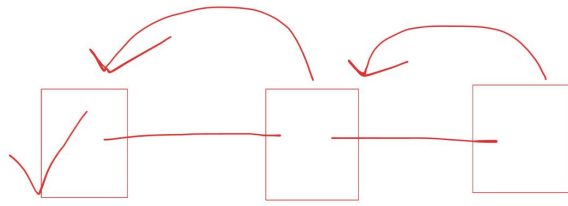
--> This works via GitHub Webhooks

Steps to implement GitHub webhook:

- 1) Go to the GitHub and select the repository for which you want to generate a build.
- 2) At the top you will be getting all these options: Code, Pull requests, Actions, Projects, Wiki, Security, Insights, Settings. Select Settings.
- 3) In the sidebar of Settings you will find an option called as "Webhooks" select that option and click on "Add Webhooks". It will ask for verification through email, verify through email.
- 4) Here you will find one option called as "Payload URL" remove the existing URL and paste the URL from Jenkins.
- 5) For getting the URL from Jenkins, login to your AWS and create one Jenkins instance and copy the public IP of Jenkins instance and paste it in a new tab along with the port number 8080 in this format:
`http://13.208.244.230:8080`
this will open Jenkins in that tab.
- 6) In the top right corner you have an option called as "Manage Jenkins" click on that. Once you are inside, under "System Configuration" select the "System" option. If you scroll down, you will find Jenkins URL copy that URL and paste it in the "Payload URL" section of GitHub in this format:
`http://13.208.244.230:8080/github-webhook/`
- 7) In "Content type" select "application/json"
- 8) If you scroll down you will get a section like this "Which events would you like to trigger this webhook?" here select "Send me everything." option
- 9) Finally click on "Add webhook"
- 10) Copy the public IP of Jenkins instance and paste it in a new tab along with the port number 8080 in this format: `http://13.208.244.230:8080`
this will open Jenkins in that tab.
- 11) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Freestyle project" then click on OK.
- 12) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/2894-vaishu/java.git>
- 13) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 14) Scroll down and you will get a section called as "Triggers" under that select "GitHub Hook Trigger for GIT SCM Polling".
- 15) Scroll down and under "Build Steps" click on Add build step under that choose "Execute Shell" option. Then write these two commands:
`javac Test.java` —> This command is used for compiling java project
`java Test.java` —> This command is used for executing java project
- 16) Scroll down and finally click on Apply and then click on Save.
- 17) There is no need to click on "Build Now" it will automatically generate a build based on the changes in GitHub.
- 18) Note that public IP of the instance should match the URL which you give for "Payload URL".

4. Build after other projects are built

--> This trigger allows one Jenkins job to automatically start after another job.



Steps to run the build using Build After Other Projects Are Built in Windows:

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name as "Project-1" and below select the option called as "Freestyle project or Maven project" then click on OK.
- 3) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/2894-vaishu/java.git>
- 4) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 7) Scroll down and under "Build Steps" click on Add build step under that choose "Execute Windows batch command" option. Then write these two commands:
`javac Test.java` → This command is used for compiling java project
`java Test.java` → This command is used for executing java project
- 8) Scroll down and finally click on Apply and then click on Save.
- 9) Next create another project by selecting the option called as "New Item". Give a job name as "Project-2" and below select the option called as "Freestyle project or Maven project" then click on OK.
- 10) Give a description, scroll down and under "Source Code Management" select Git. It will ask for Repository URL paste the GitHub repository URL <https://github.com/2894-vaishu/mavenproject.git>
- 11) Scroll down and under "Branches to Build" specify the branch name as master or main based on the GitHub repository.
- 12) Scroll down and you will get a section called as "Triggers" under that select "Build after another projects are built". As soon as you select this it will ask you to write the project name, select "Project-1".
- 13) Scroll down and under "Build Steps" click on Add build step under that choose "Execute Windows batch command" option. Then write these two commands: `mvn package`
- 14) Scroll down and finally click on Apply and then click on Save.
- 15) Go to Project-1 and click on "Build Now". You can see a build is being generated and in the end you can see that it will generate the build for Project-2 as well.

Role-Based Authorization in Jenkins

- > Role-Based Authorization in Jenkins is a security system that lets you control who can access or modify different parts of Jenkins.
- > Instead of giving everyone full control, you assign specific permissions through roles.
- > Each role defines what a user or group of users can do, such as creating jobs, building pipelines, managing nodes, or viewing results.
- > It ensures that people only have access to what they need.

Why Jenkins Needs Role-Based Authorization:

- > By default, Jenkins either allows everyone to access it freely or restricts everything to administrators.
- > This is fine for learning or a single-user setup but becomes unsafe in a shared environment.
- > In a company or team project, multiple users work on Jenkins simultaneously.
- > Without access control, anyone could delete jobs, expose credentials, or break pipelines.
- > Role-Based Authorization fixes this by allowing fine-grained control.
- > It is essential for security, order, and accountability in multi-user Jenkins servers.

How It Works:

- > Jenkins uses a plugin called "Role-Based Authorization Strategy."
- > Once installed, you can define different levels of access through roles.
- > Roles can be applied globally across Jenkins or only to specific jobs or nodes.
- > After defining roles, you assign users to those roles.
- > When users log in, they can only perform the actions allowed by their assigned role.

Steps to assign role based authorization:

- 1) Copy the public ip of jenkins instance and paste it in a new tab along with the port number 8080 in this format: <http://13.208.244.230:8080> this will open Jenkins in that tab.
- 2) Once you are inside Jenkins, Go to Manage Jenkins option. Under "System Configuration" select "Plugins". under Available plugins, search "Role-based Authorization Strategy" and install it.
- 3) Then Go back to Manage Jenkins, and this time under "Security" select "Security". Under "Security Realm" select "Jenkins' own user database" and under "Authorization" select "Role-Based Strategy". Scroll down till the end and click on Apply and then click on Save.
- 4) Next under "Security" select "Manage and Assign Roles" option and then under "Role to add" add the role name write Developer for example (Roles --- Developer, Test engineer, DevOps enggineer etc ...) and click on Add.
- 5) You will see that as soon as you click on Add, under "Global Roles" Developer option is created. Here Tick on these: Overall: Read, Job: Build, Configure, Create, Read, Workspace. These permissions are enough for a Developers.
- 6) To create the users for the Developers role go to "Manage Jenkins" and under "Security", select "Users". then click on "Create User". It will ask for the Username, Password, e-mail Fill all the credentials for the users for who you want to assign the developer role.
- 7) Now that the user is created go to "Manage Jenkins" option and under "Security" select "Manage and Assign Roles" option and in the sidebar you can see an option called as "Assign roles" click on it. Here you have a button called as "Add user" click on it. It will ask you for UserID, give the username and then tick on user based on his/her role (Developer). Finally scroll down and click on Apply and Save.
- 8) To cross verify, logout from your account and log in to the account by using credentials of the user you have created and then verify the permissions you gave under roles are working or not.

3) Pipeline Jobs and its types:

- > A Pipeline Job in Jenkins is a type of job that allows you to define your entire build process using a Jenkinsfile — a script written in Groovy-based DSL (Domain Specific Language).
- > This file describes stages, steps, and conditions that define how your CI/CD pipeline runs.
- > It turns your build process into code, often called Pipeline as Code.
- > Basically it is a script to perform sequence of steps which will perform tasks such as Building, testing and deploying.

Why Pipelines?

Traditional jobs (like Freestyle or Maven) are limited — you click buttons to configure steps manually.

Pipeline jobs, however:

- > Are automated and repeatable
- > Can be version controlled (Jenkinsfile stored in Git)
- > Handle complex workflows like parallel builds, multiple environments, approvals, etc.
- > Survive Jenkins restarts (thanks to durability and checkpoints)
- > Are portable between Jenkins instances

Types of Pipelines:

- 1) Declarative Pipeline
- 2) Scripted Pipeline

1. Declarative Pipeline:

- > A Declarative Pipeline in Jenkins is a simplified and structured way to define your Continuous Integration/Continuous Deployment (CI/CD) process using a predefined syntax.
 - > It is designed to make pipeline creation easier and more readable.
 - > Declarative pipelines use the keyword pipeline {} as the root block, and have a defined hierarchy with specific sections such as agent, stages, steps, post, etc.
 - > They are preferred for most use cases because they are easier to read, less error-prone, and automatically handle restarts.
- A Declarative Pipeline in Jenkins follows a strict structure. Every valid pipeline starts with a pipeline {} block and contains several optional sections. Here's the general structure:

The Structure of a Declarative Pipeline

```
pipeline {  
  agent any  
  environment { }  
  parameters { }  
  options { }  
  triggers { }  
  tools { }  
  stages {  
    stage('Example') {  
      steps {  
        script { }  
      }  
    }  
  }  
  post {  
    success { }  
    failure { }  
    always { }  
  }  
}
```

Explanation of Each Block

1. pipeline

- > This is the root block of any Jenkins Declarative Pipeline.
- > All other sections must be defined inside this block.
- > It defines the beginning and end of the pipeline.
- > Without this block, Jenkins will not recognize the file as a declarative pipeline.

Example:

```
pipeline {  
  // all inner blocks go here  
}
```

2. agent

- > This block defines where the pipeline or a stage will run.
- > It tells Jenkins which node, machine, or environment to use to execute the job.

Common Options:

- > agent any → Run on any available Jenkins agent.
- > agent none → Used if you want to specify agent separately inside each stage.
- > agent { label 'linux' } → Run on a node with a specific label.
- > agent { docker 'maven:3.9.0' } → Run inside a Docker container.

3. environment

- > This block is used to define environment variables that are available throughout the pipeline.
- > Environment variables store reusable values such as file paths, credentials, or application names.
- > You can use these later as `${APP_NAME}` or `${env.APP_NAME}`.

Example:

```
environment {  
    APP_NAME = "spring-petclinic"  
    BUILD_DIR = "target"  
}
```

4. parameters

- > This block allows users to provide input to the pipeline before it runs.
- > It is useful when you want to build different branches, versions, or configurations using the same Jenkinsfile.
- > This creates a text box on the Jenkins job page where you can specify a branch name before running.

Example:

```
parameters {  
    string(name: 'BRANCH', defaultValue: 'main', description: 'Branch to build from')  
}
```

5. options

- > This block defines pipeline-wide settings that control behavior such as timeouts, build history retention, and concurrency.

Common options:

- > timeout(time: 10, unit: 'MINUTES') – Stops the build if it runs longer than 10 minutes.
- > buildDiscarder(logRotator(numToKeepStr: '5')) – Keeps only the last 5 builds.
- > disableConcurrentBuilds() – Prevents overlapping builds.

Example:

```
options {  
    timeout(time: 15, unit: 'MINUTES')  
    buildDiscarder(logRotator(numToKeepStr: '5'))  
}
```

6. triggers

- > This block defines automatic build triggers, meaning when and how Jenkins should automatically start this job.

Common Triggers:

- > pollSCM('*/* 5 * * *') – Poll the Git repository every 5 minutes for changes.
- > cron('0 2 * * 1-5') – Schedule builds using a cron expression (e.g., every weekday at 2 AM).
- > upstream('OtherJob') – Trigger when another Jenkins job completes.

Example:

```
triggers {  
    pollSCM('*/* 5 * * *')  
}
```

7. tools

- > This block declares auto-installed tools (like JDK, Maven, or Git) that Jenkins should use.
- > The tools must already be configured in Manage Jenkins → Global Tool Configuration.
- > This ensures that Jenkins uses the specified JDK and Maven versions for the build.

Example:

```
tools {  
    jdk 'JDK11'  
    maven 'Maven3'  
}
```

8. stages

- > This block contains all the stages of the pipeline.
- > Each stage represents a major phase of the build, such as "Build", "Test", "Deploy".
- > Organizes your CI/CD process into logical steps and provides visual clarity in Jenkins.

Example:

```
stages {  
  stage('Build') {  
    steps {  
      echo 'Building the project...'  
    }  
  }  
}
```

9. stage

- > A stage is a logical section within stages.
- > Each stage can have multiple steps that define the commands to execute.
- > It is used to separate tasks like building, testing, packaging, and deploying.

Example:

```
stage('Build') {  
  steps {  
    bat 'mvn clean package'  
  }  
}
```

10. steps

- > This block defines the actual commands or tasks Jenkins will execute inside a stage.

Examples of steps:

- > bat – Run Windows batch commands.
- > sh – Run shell commands on Linux/Mac.
- > git – Clone a Git repository.
- > archiveArtifacts – Save build artifacts in Jenkins.

Example:

```
steps {  
  bat 'mvn clean package'  
}
```

11. script

- > This block allows custom Groovy scripting inside a Declarative Pipeline.
- > Useful when you need logic such as if, for, or variable manipulation.

Example:

```
script {  
  if (env.BRANCH_NAME == 'main') {  
    echo 'This is the main branch build.'  
  }  
}
```

12. post

- > This block defines actions that happen after the pipeline finishes, regardless of success or failure.
- > You can place this block at the pipeline level or inside an individual stage.

Subsections:

- > success – Runs only if the build succeeds.
- > failure – Runs only if the build fails.
- > always – Runs every time (cleanup or notifications).

Example:

```
post {  
  success {  
    echo 'Build succeeded!'  
  }  
  failure {  
    echo 'Build failed.'  
  }  
  always {  
    echo 'Cleaning up...'  
    cleanWs()  
  }  
}
```

Steps to implement declarative pipeline

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Pipeline" then click on OK.
- 3) Give a description, scroll down and under "Pipeline" you have to write a script. Instead of writing the script choose the option "Hello world" from the right corner. It will provide you a sample script.
- 4) If you want to clone a GitHub repository then click on the "Pipeline Syntax" link. It will open a new tab where under "Sample Step" choose git:Git
- 5) Paste the URL of the repository: <https://github.com/Sumukha47/java.git>
- 6) Then verify the branch as master or main and then click on "Generate Pipeline Script". It will give you a readymade command: `git 'https://github.com/Sumukha47/java.git'`
- 7) Copy that command and paste it under stage('Git-Clone') and steps like this:

```
pipeline {
    agent any

    stages {
        stage('Git-Clone') {
            steps {
                git 'https://github.com/Sumukha47/java.git'
            }
        }
        stage('Java-Project') {
            steps {
                bat '''javac Test.java
                    java Test.java'''
            }
        }
    }
}
```

- 8) Now if you want to execute java commands then once again click on "Pipeline Syntax" link. It will open a new tab where under "Sample Step" choose "bat: Windows Batch Script". Under "Batch Script" write down these commands:

```
javac Test.java
java Test.java
```

After writing these commands, click on "Generate Pipeline Script" it will give you a command, copy the command and paste it as shown in the above script. The command looks like this:

```
bat '''javac Test.java
    java Test.java'''
```

- 9) Scroll down and finally click on Apply and then click on Save.
- 10) In the side bar you have an option called as "Build Now" click on that. You can see a build is being generated click on that once after it is generated.

2. Scripted Pipeline:

- > A Scripted Pipeline is written entirely in Groovy syntax.
- > Unlike Declarative Pipelines (which follow a fixed structure), scripted pipelines give maximum control over build flow using standard Groovy programming constructs like:
 - > Variables
 - > Loops
 - > Conditions (if, else)
 - > Functions (methods)
 - > Exception handling (try, catch)

The Structure of a Scripted Pipeline

```
node('agent-label') {  
  
    // 1. PARAMETERS (accessed using params.PARAM_NAME)  
    // 2. ENVIRONMENT VARIABLES  
    // 3. TOOL SETUP  
    // 4. BUILD TRIGGERS  
    // 5. MAIN LOGIC INSIDE TRY-CATCH-FINALLY  
    // 6. FAILURE HANDLING  
    // 8. END MESSAGE  
  
}
```

Explanation of Each Block

1. node

- > The node block allocates an executor (agent) on a Jenkins node (where the build will run).
- > All actual build steps are written inside this block.
- > It acts like the "workspace" where all operations occur.

Example:

```
node('windows') {  
    echo "Running on Windows node"  
}
```

2. stage Block

- > A logical division of your pipeline — each stage represents one phase (e.g., Checkout, Build, Test, Deploy).
- > It is used for visualization and separation of steps in Jenkins UI.

Example:

```
stage('Test') {  
    echo "Running unit tests..."  
}
```

3. environment Variables

- > You can define variables to be reused throughout your pipeline.
- > Stores data like credentials, paths, or configuration that multiple stages use.

Example:

```
env.BRANCH_NAME = "main"  
echo "Building branch ${env.BRANCH_NAME}"
```

4. Parameters

- > Used to get user input before the job starts (e.g., branch name, build type).
- > It makes the pipeline configurable at runtime.

Syntax:

```
properties([  
    parameters([  
        string(name: 'PARAM1', defaultValue: 'default', description: 'Example parameter'),  
        booleanParam(name: 'DEPLOY', defaultValue: true, description: 'Deploy after build')  
    ])  
])
```

Example:

```
echo "Parameter received: ${params.PARAM1}"
```

5. Tools

- > Automatically installs and configures required tools (e.g., JDK, Maven, Git).
- > It simplifies environment setup.

Syntax:

```
def mvnHome = tool name: 'Maven3', type: 'maven'
```

Example:

```
def javaHome = tool name: 'JDK11', type: 'jdk'
env.PATH = "${javaHome}/bin:${env.PATH}"
```

6. Triggers

- > Used to schedule automatic build execution.
- > It automates the pipeline (e.g., daily builds, SCM polling).

Syntax:

```
properties([
    pipelineTriggers([
        cron('H 2 * * 1-5'),    // Run every weekday at 2 AM
        pollSCM('* * * * *')    // Poll every minute
    ])
])
```

7. try-catch-finally (Post Actions)

- > Scripted Pipelines do not have a post {} block like Declarative Pipelines.
- > Instead, we handle success, failure, and always conditions using try-catch-finally.
- > Control flow management and handling notifications.

Syntax:

```
try {
    // Normal build steps
} catch (err) {
    // Runs on failure
} finally {
    // Runs always (cleanup, notifications)
}
```

Steps to implement scripted pipeline

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Pipeline" then click on OK.
- 3) Give a description, scroll down and under "Pipeline" you have to write a script. Instead of writing the script choose the option "Scripted Pipeline" from the right corner. It will provide you a sample script. Modify the sample script however you want.
- 4) If you want to clone a GitHub repository then click on the "Pipeline Syntax" link. It will open a new tab where under "Sample Step" choose git:Git
- 5) Paste the URL of the repository: <https://github.com/Sumukha47/java.git>
- 6) Then verify the branch as master or main and then click on "Generate Pipeline Script". It will give you a readymade command: `git 'https://github.com/Sumukha47/java.git'`
- 7) Copy that command and paste it under stage('Git-Clone') and steps like this:

```
node {
    stage('Git-clone') {
        git 'https://github.com/Sumukha47/java.git'
    }
    stage('java-commands') {
        bat '''javac Test.java
              java Test.java'''
    }
}
```

- 8) Now if you want to execute java commands then once again click on "Pipeline Syntax" link. It will open a new tab where under "Sample Step" choose "bat: Windows Batch Script". Under "Batch Script" write down these commands:

```
javac Test.java
java Test.java
```

After writing these commands, click on "Generate Pipeline Script" it will give you a command, copy the command and paste it as shown in the above script. The command looks like this:

```
bat '''javac Test.java
      java Test.java'''
```

- 9) Scroll down and finally click on Apply and then click on Save.
- 10) In the side bar you have an option called as "Build Now" click on that. You can see a build is being generated click on that once after it is generated.

Example script for sample maven project:

```
node {  
  
    def gitrepo = 'https://github.com/Sumukha47/mavenproject.git'  
    def stages = ['compile','test','package']  
  
    stage('Git-clone'){  
        git gitrepo  
    }  
    for (i in stages)  
    {  
        stage (i)  
        {  
            if (i == 'compile')  
            {  
                bat 'mvn compile'  
            }  
            if (i == 'test')  
            {  
                bat 'mvn test'  
            }  
            if (i == 'package')  
            {  
                bat 'mvn package'  
            }  
        }  
    }  
}
```

Steps to enable email notification in Jenkins

- 1) Open a browser and search localhost:8080 it will open Jenkins dashboard.
- 2) In the sidebar you have an option called as "New Item" click on it. Give a job name and below select the option called as "Pipeline" then click on OK.
- 3) Give a description, scroll down and under "Pipeline" you have to write a script. Instead of writing the script choose the option "Hello world" from the right corner. It will provide you a sample script.
- 4) If you want to clone a GitHub repository then click on the "Pipeline Syntax" link. It will open a new tab where under "Sample Step" choose git:Git
- 5) Paste the URL of the repository: <https://github.com/2894-vaishu/java.git>
- 6) Then verify the branch as master or main and then click on "Generate Pipeline Script". It will give you a ready-made command: `git 'https://github.com/2894-vaishu/java.git'`
- 7) Copy that command and paste it under stage('Git-Clone') and steps like this:

```
pipeline {
    agent any

    stages {
        stage('Git-Clone') {
            steps {
                git 'https://github.com/2894-vaishu/java.git'
            }
        }
        stage('Java-Project') {
            steps {
                bat '''javac Test.java
                    java Test.java'''
            }
        }
    }
}
```

- 8) Now if you want to execute java commands then once again click on "Pipeline Syntax" link. It will open a new tab where under "Sample Step" choose "bat: Windows Batch Script". Under "Batch Script" write down these commands:

```
javac Test.java
java Test.java
```

After writing these commands, click on "Generate Pipeline Script" it will give you a command, copy the command and paste it as shown in the above script. The command looks like this:

```
bat '''javac Test.java
    java Test.java'''
```

- 9) Scroll down and finally click on Apply and then click on Save.
- 10) Now that you have created a declarative pipeline, we need to enable e-mail notification for the successful build we have created. For that go to manage Jenkins and install "Email Extension Template Plugin".

- 11) Go to your gmail and at the top right corner you can see your account logo. click on it and select "Manage your Google account".
- 12) After this in the sidebar you will find an option called as "Security" select that and scroll down and make sure "2 step verification is enabled". If it is not enabled, enable it.
- 13) Next search for "App passwords" in the searchbar and select it.
- 14) Under "To create a new app specific password, type a name for it below..." write the name as Jenkins and then click on create.
- 15) You will get a password copy and paste it somewhere in your machine.
- 16) Now once again go back to Jenkins and go to manage Jenkins. Then under "System Configuration" select "System" scroll down till the end and you will find an option called as "E-mail Notification". Under this option, you will find "SMTP server" write `smtp.gmail.com`
- 17) Then click on "Advanced" and tick on "Use SMTP Authentication" and give your e-mail. Next paste the password which you have saved. Tick on "Use SSL". Under "SMTP Port" give 465. Then tick on "Test configuration by sending test e-mail" and give your e-mail and then click on test configuration. Check whether the e-mail is sent or not.
- 18) Go to the declarative-pipeline which you have created and click on configure and then under script add a post script after stages like one given in example.
- 19) To get this part: `mail bcc: "`

```
body: 'This is to inform you that the build is successful',
cc: 'ankan.sardar@qspiders.in',
from: 'vaishnavi.g@qspiders.in',
replyTo: "",
subject: 'Build Success',
to: 'sumukha.u@gmail.com'
```

Click on "Pipeline Syntax". It will open a new tab where under "Sample step" choose "mail:Mail" and specify for whom you want to send the mail (to, cc, bcc, subject, body) and finally click on "Generate pipeline script". You will be getting a script. copy and paste it under post ----> success block and also make necessary changes like given in the example.

Note:

- > To configure Jenkins to automatically install the necessary Java and Maven versions install a plugin called as "Eclipse Temurin installer Plugin". Depending on your Jenkins version — this plugin adds "Install from adoptium.net" option. Same works for Maven.
- > If you want your Build_stamp to be used while writing e-mail notification, Install a plugin called as "Build Timestamp plugin".